



HUMBER

Faculty of Applied Sciences & Technology

Course Number	MENG-2500
Course Title	Mechatronics Project
Semester/Year	2025

Lab/Tutorial Report No.	Final Report
Title of the Experiment	Final Project Report
Instructor Name	Mehrdad Tabrizipour
Section No.	ONB
Submission Date	4/22/2025
Due Date	4/22/2025

Student Name	ID
TANVI BHANDERI	N01605037
GURSHAN SINGH	N01604467
HRISHIKESH PATEL	N01609756
ASSADULLAH JIN	N01626920
SOKRAT KRISTULLA	N01655524

* By signing above, you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a ZERO on the work or possibly more severe penalties.

<https://academic-regulations.humber.ca/2021-2022/17.0-ACADEMIC-MISCONDUCT>

Introduction

In manufacturing, automation is essential for increasing productivity and lowering human error. The goal of this project is to use embedded microcontroller devices to automate the box cartooning process. It targets manufacturers on a budget that are searching for a dependable and scalable solution.

Abstract

In this project, an automated cartooning machine that uses embedded technology is designed and put into operation. The objective is to create a small-scale, affordable solution for manufacturers. Pneumatic cylinders, a keypad, an LCD, sensors, servo motors, and a stepper motor are all controlled by a TM4C123GH6PM microcontroller. It has four operating phases and can automatically fold and lock box flaps. A closed-loop control system is used by the machine to ensure accuracy and dependability.

Problem Overview and Scope Statement

Manual box folding is labor-intensive, inconsistent, and inefficient in small to medium-sized businesses. The goal of this project is to create a device that uses actuators and sensor feedback to autonomously fold and lock box flaps. The solution needs to be small, flexible, and reasonably priced (within a \$300 budget).

CODE

```
1  #include "TM4C123GH6PM.h"
2  #include <stdint.h>
3
4  // DCV control using PB0, PB1, PB5
5  #define DCV1 0x01 // PB0
6  #define DCV2 0x02 // PB1
7  #define DCV3 0x20 // PB5
8
9  // Stepper motor pins
10 #define STEP_PIN 0x40 // PB6
11 #define DIR_PIN 0x80 // PB7
12 #define DEG_PER_STEP 0.45 // For 400 steps/rev
13
14 #define DEFAULT_DELAY 32
15
16 int delay_time = DEFAULT_DELAY;
17
18 // Stepper and DCV Init
19 void PortB_Init(void) {
20     SYSTCL->RCGCGPIO |= 0x02;
21     while ((SYSTCL->PRGPIO & 0x02) == 0);
22
23     GPIOB->LOCK = 0x4C4F434B;
24     GPIOB->CR |= 0xFF;
25     GPIOB->AMSEL &= ~0xFF;
26     GPIOB->PCTL &= ~0xFFFFFFFF;
27     GPIOB->DIR |= 0xFF;
28     GPIOB->AFSEL &= ~0xFF;
29     GPIOB->DEN |= 0xFF;
30 }
31
32
33
34 // Accurate microsecond delay using Timer1A
35 void Delay_MicroSecond(int time) {
36     int i;
37     SYSTCL->RCGCTIMER |= 2;
38     TIMER1->CTL = 0;
39     TIMER1->CFG = 0x04;
40     TIMER1->TAMR = 0x02;
41     TIMER1->TAILR = 16 - 1;
42     TIMER1->ICR = 0x1;
43     TIMER1->CTL |= 0x01;
44
45     for (i = 0; i < time; i++) {
46         while ((TIMER1->RIS & 0x1) == 0);
```

```

47     TIMER1->ICR = 0x1;
48 }
49 }
50
51 // Millisecond delay
52 void Delay_ms(int ms) {
53     int i, j;
54     for (i = 0; i < ms; i++)
55         for (j = 0; j < 3180; j++);
56 }
57
58 // Servo pulse function
59 void Servo_Set_Angle(int pulseWidth, uint8_t pin) {
60     for (int i = 0; i < 50; i++) {
61         GPIOB->DATA |= (1 << pin);
62         Delay_MicroSecond(pulseWidth);
63         GPIOB->DATA &= ~(1 << pin);
64         Delay_MicroSecond(20000 - pulseWidth);
65     }
66 }
67
68 // Degree to pulse width
69 int AngleToPulse(int degree) {
70     return 1000 + ((degree * 1800) / 180);
71 }
72
73 // GPIOA (Encoder) and GPIOF (Debug LEDs)
74 void GPIO_Init(void){
75     SYSCCTL->RCGCGPIO |= 0x21;
76     while((SYSCCTL->PRGPIO & 0x21) == 0);
77
78     GPIOA->DIR &= ~(0x38);
79     GPIOA->DEN |= 0x38;
80
81     GPIOF->DIR |= 0x06;
82     GPIOF->DEN |= 0x06;
83 }
84
85 // Port D for pneumatic valves
86 void PortD_Init(void) {
87     SYSCCTL->RCGCGPIO |= 0x08;
88     GPIOD->LOCK = 0x4C4F434B;
89     GPIOD->CR |= 0x07;
90     while ((SYSCCTL->RCGCGPIO & 0x08) == 0);
91
92     GPIOD->DIR |= 0x07;

```

```

93     GPIOD->DEN |= 0x07;
94 }
95
96
97
98 // DCV1 = PB4, DCV2 = PB1, DCV3 = PB5 (active-low)
99 void DCV_Set(uint8_t dcv_mask) {
100     uint8_t mask = (1 << 4) | (1 << 1) | (1 << 5); // PB4, PB1, PB5
101     GPIOB->DATA = (GPIOB->DATA & ~mask) | (~dcv_mask & mask);
102 }
103
104
105
106 // Stepper motor rotation (replaces encoder-based function)
107 void RotateMotor(float angle, uint32_t speed_delay_ms, int direction) {
108     uint32_t steps = (uint32_t)(angle / DEG_PER_STEP);
109
110     if (direction == 1)
111         GPIOB->DATA |= DIR_PIN; // CW
112     else
113         GPIOB->DATA &= ~DIR_PIN; // CCW
114
115     for (uint32_t i = 0; i < steps; i++) {
116         GPIOB->DATA |= STEP_PIN;
117         Delay_ms(speed_delay_ms);
118         GPIOB->DATA &= ~STEP_PIN;
119         Delay_ms(speed_delay_ms);
120     }
121 }
122
123 // Optional manual fine-adjust
124 void Manual_Adjust_Steps(int steps) {
125     for (int i = 0; i < steps; i++) {
126         GPIOB->DATA |= STEP_PIN;
127         Delay_ms(delay_time);
128         GPIOB->DATA &= ~STEP_PIN;
129         Delay_ms(delay_time);
130     }
131 }
132
133
134 int main(void) {
135     PortB_Init();
136     GPIO_Init();
137
138     DCV_Set(0x20); // DCV3 ON

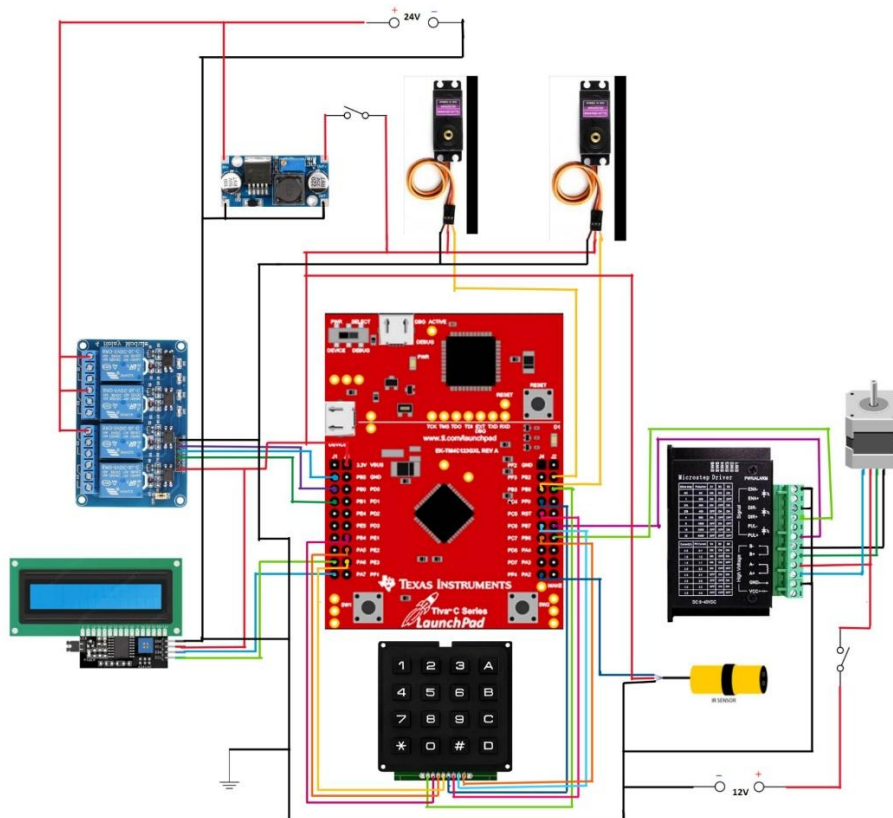
```

```

139     Delay_ms(500);
140
141     Servo_Set_Angle(AngleToPulse(140), 2);
142     Servo_Set_Angle(AngleToPulse(0), 3);
143
144     while (1) {
145         Servo_Set_Angle(AngleToPulse(0), 2);
146         Delay_ms(500);
147         Servo_Set_Angle(AngleToPulse(100), 3);
148         DCV_Set(0x22);           // DCV2 + DCV3 ON
149         Delay_ms(500);
150
151         DCV_Set(0x12);           // DCV1 + DCV2 ON
152         Delay_ms(2000);
153
154         DCV_Set(0x10);           // DCV1 ON only
155         Servo_Set_Angle(AngleToPulse(0), 3);
156         RotateMotor(67.5, 16, 1);
157
158         Servo_Set_Angle(AngleToPulse(140), 2);
159         DCV_Set(0x20);           // DCV3 ON
160
161         Delay_ms(1000);
162         RotateMotor(67.5, 16, 1);
163         Delay_ms(500);
164     }
165

```

CIRCUIT SCHEMATIC



1. Electrical Components

TIVA C LaunchPad (TM4C123GXL)

Function: Microcontroller board used for logic and control

Datasheet: <https://www.ti.com/lit/ds/symlink/tm4c123gh6pm.pdf>

Function: Controls solenoid valves and vacuum generator

Datasheet: <https://www.circuitbasics.com/wp-content/uploads/2015/11/SRD-05VDC-SL-C-Datasheet.pdf>

MG996R Servo Motor

Function: Used for flap folding mechanism

Datasheet: <https://www.electronicoscaldas.com/datasheet/MG996R Tower-Pro.pdf>

4x4 Matrix Keypad

Function: Used for inputting box count or control signals

Datasheet: <https://cdn.sparkfun.com/assets/f/f/a/5/0/DS-16038.pdf>

I2C LCD Display (16x2)

Function: Displays status and user feedback

Datasheet: https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf

NEMA 17 Stepper Motor

Function: Used for pushing or rotating boxes

Datasheet: <https://pages.pbclinear.com/rs/909-BFY-775/images/Data-Sheet-Stepper-Motor-Support.pdf>

Stepper Motor Driver (DM542)

Function: Drives the NEMA 17 motor

Datasheet: <https://kitaez-cnc.com/f/dm542.pdf>

Ultrasonic Sensor (HC-SR04)

Function: Detects object presence or distance

Datasheet: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>

LM2596 Buck Converter

Function: Steps down voltage to 5V for low-power components

Datasheet: <https://www.ti.com/lit/ds/symlink/lm2596.pdf>

2. Pneumatic Components

Double-Acting Cylinders

Function: Used for pickup and ejection. Bore: 0.75in, Rod: 0.25in, Stroke: 4in

Datasheet: https://ngsgi.com/pdfs/cip_da_cylinders_specs.pdf

5/2 Solenoid Valve (Spring Return)

Function: Controls airflow direction for cylinder extension/retraction

Datasheet: <https://www.uflowvalve.com/pneumatic-valve/product/5-2-single-solenoid-valve-with-spring-return>

Venturi Vacuum Generator

Function: Generates vacuum using compressed air for suction cup

Datasheet: <https://cdn.norgren.com/pdf/CylindricalVenturiVacuumGenerators.pdf>

Flat Suction Cup (25mm)

Function: Grips flat paperboard carton using vacuum

Datasheet: <https://www.vacmotion.com/Details.aspx?Cat=Suctioncups&Id=711>

Conclusion and Future Plans

This project effectively used embedded systems to create a cost-effective cartooning machine. It offers packaging applications a scalable automated option. Plans call for integrating IoT technologies for remote monitoring and using parameterized control logic to modify the design to fit different box sizes.